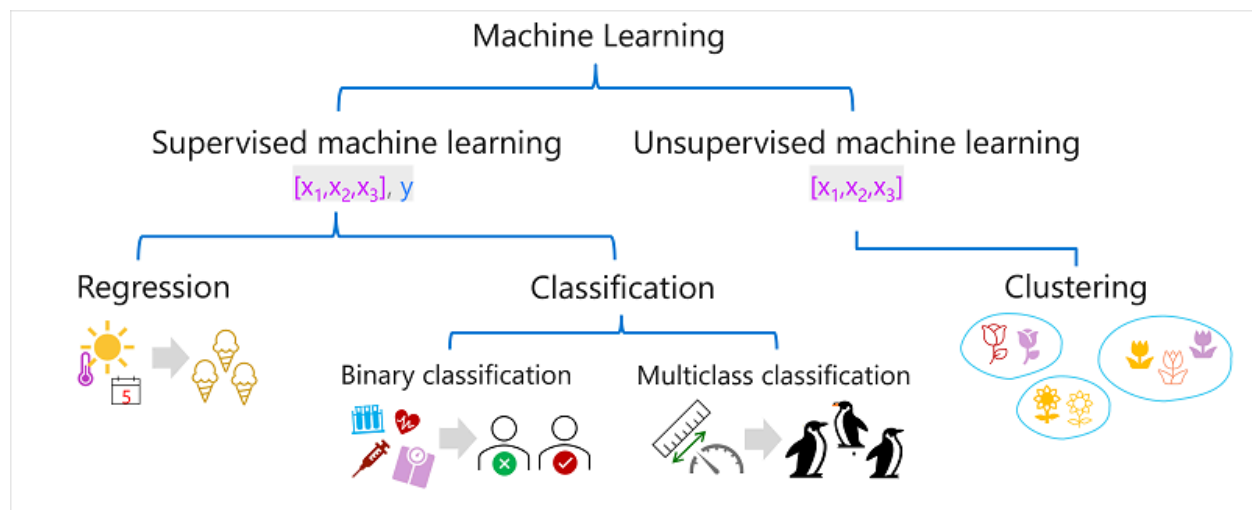


Types of machine learning

There are multiple types of machine learning, and you must apply the appropriate type depending on what you're trying to predict. A breakdown of common types of machine learning is shown in the following diagram.



Supervised machine learning

Supervised machine learning is a general term for machine learning algorithms in which the training data includes both feature values and known label values. Supervised machine learning is used to train models by determining a relationship between the features and labels in past observations, so that unknown labels can be predicted for features in future cases.

Regression

Regression is a form of supervised machine learning in which the label predicted by the model is a numeric value. For example:

- The number of ice creams sold on a given day, based on the temperature, rainfall, and windspeed.
- The selling price of a property based on its size in square feet, the number of bedrooms it contains, and socio-economic metrics for its location.
- The fuel efficiency (in miles-per-gallon) of a car based on its engine size, weight, width, height, and length.

Classification

Classification is a form of supervised machine learning in which the label represents a categorization, or class. There are two common classification scenarios.

Binary classification

In binary classification, the label determines whether the observed item is (or isn't) an instance of a specific class. Or put another way, binary classification models predict one of two mutually exclusive outcomes. For example:

- Whether a patient is at risk for diabetes based on clinical metrics like weight, age, blood glucose level, and so on.
- Whether a bank customer will default on a loan based on income, credit history, age, and other factors.
- Whether a mailing list customer will respond positively to a marketing offer based on demographic attributes and past purchases.

In all of these examples, the model predicts a binary true/false or positive/negative prediction for a single possible class.

Multiclass classification

Multiclass classification extends binary classification to predict a label that represents one of multiple possible classes. For example,

- The species of a penguin (Adelie, Gentoo, or Chinstrap) based on its physical measurements.
- The genre of a movie (comedy, horror, romance, adventure, or science fiction) based on its cast, director, and budget.

In most scenarios that involve a known set of multiple classes, multiclass classification is used to predict mutually exclusive labels. For example, a penguin can't be both a Gentoo and an Adelie. However, there are also some algorithms that you can use to train multilabel classification models, in which there may be more than one valid label for a single observation. For example, a movie could potentially be categorized as both science fiction and comedy.

Unsupervised machine learning

Unsupervised machine learning involves training models using data that consists only of feature values without any known labels. Unsupervised machine learning algorithms determine relationships between the features of the observations in the training data.

Clustering

The most common form of unsupervised machine learning is clustering. A clustering algorithm identifies similarities between observations based on their features, and groups them into discrete clusters. For example:

- Group similar flowers based on their size, number of leaves, and number of petals.

- Identify groups of similar customers based on demographic attributes and purchasing behavior.

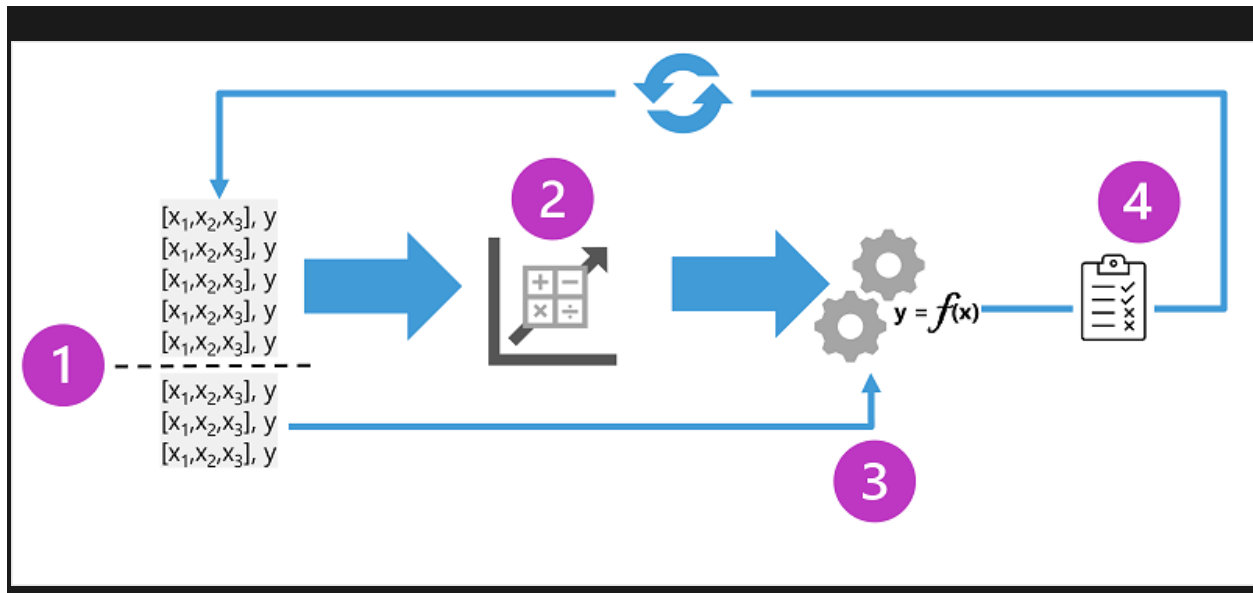
In some ways, clustering is similar to multiclass classification; in that it categorizes observations into discrete groups. The difference is that when using classification, you already know the classes to which the observations in the training data belong; so the algorithm works by determining the relationship between the features and the known classification label. In clustering, there's no previously known cluster label and the algorithm groups the data observations based purely on similarity of features.

In some cases, clustering is used to determine the set of classes that exist before training a classification model. For example, you might use clustering to segment your customers into groups, and then analyze those groups to identify and categorize different classes of customer (high value - low volume, frequent small purchaser, and so on). You could then use your categorizations to label the observations in your clustering results and use the labeled data to train a classification model that predicts to which customer category a new customer might belong.

Regression

Regression models are trained to predict numeric label values based on training data that includes both features and known labels. The process for training a regression model (or indeed, any supervised machine learning model) involves multiple iterations in which you use an appropriate algorithm (usually with some parameterized settings) to train a model, evaluate the model's predictive performance, and refine the model by

repeating the training process with different algorithms and parameters until you achieve an acceptable level of predictive accuracy.



The diagram shows four key elements of the training process for supervised machine learning models:

1. Split the training data (randomly) to create a dataset with which to train the model while holding back a subset of the data that you'll use to validate the trained model.
2. Use an algorithm to fit the training data to a model. In the case of a regression model, use a regression algorithm such as linear regression.
3. Use the validation data you held back to test the model by predicting labels for the features.
4. Compare the known actual labels in the validation dataset to the labels that the model predicted. Then aggregate the differences between the predicted and actual label values to calculate a metric that indicates how accurately the model predicted for the validation data.

After each train, validate, and evaluate iteration, you can repeat the process with different algorithms and

parameters until an acceptable evaluation metric is achieved.

Example - regression

Let's explore regression with a simplified example in which we'll train a model to predict a numeric label (y) based on a single feature value (x). Most real scenarios involve multiple feature values, which adds some complexity; but the principle is the same.

For our example, let's stick with the ice cream sales scenario we discussed previously. For our feature, we'll consider the *temperature* (let's assume the value is the maximum temperature on a given day), and the label we want to train a model to predict is the number of ice creams sold that day. We'll start with some historic data that includes records of daily temperatures (x) and ice cream sales (y):



Temperature (x)	Ice cream sales (y)
51	1
52	0
67	14
65	14
70	23
69	20
72	23
75	26
73	22
81	30

78	26
83	36

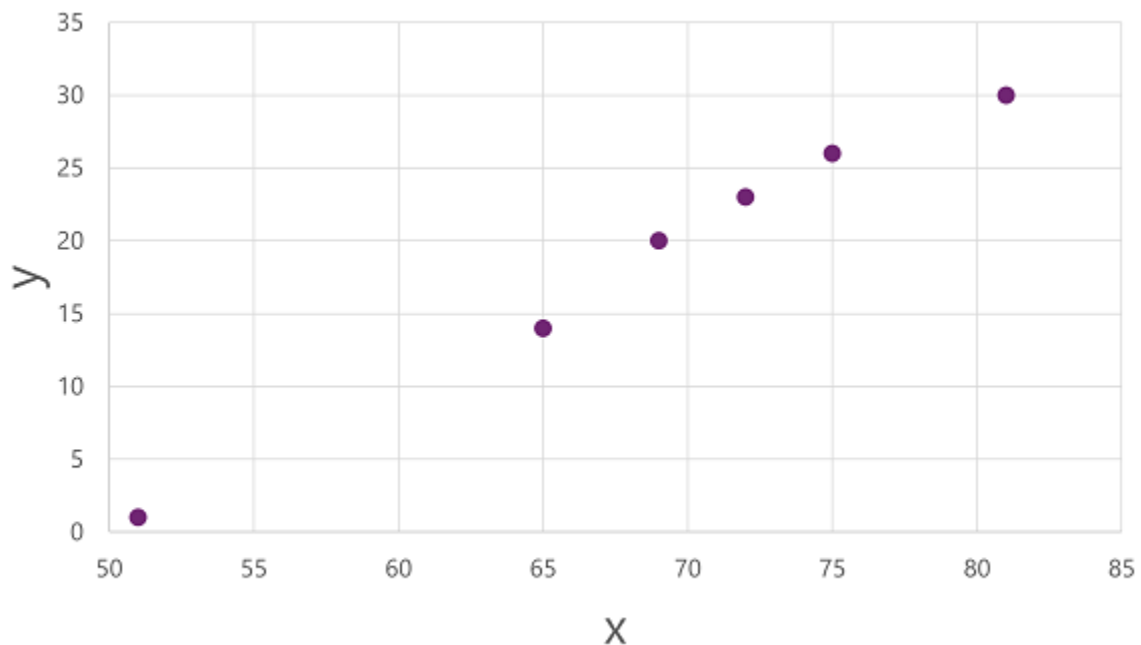
Training a regression model

We'll start by splitting the data and using a subset of it to train a model. Here's the training dataset:

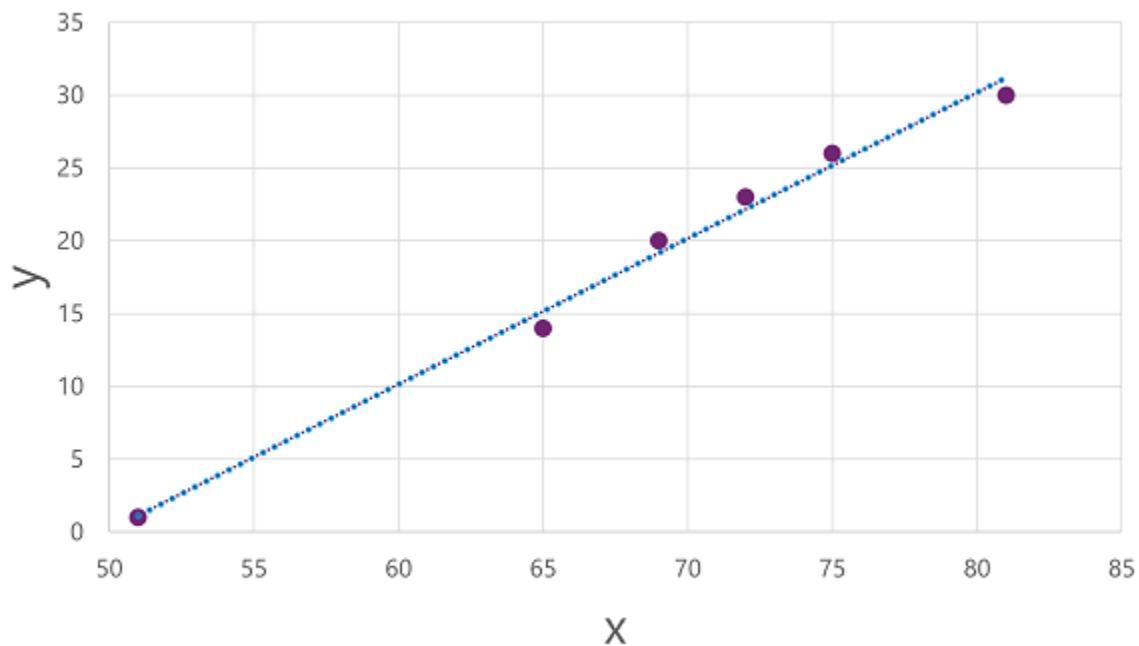
Temperatur Ice cream

e (x)	sales (y)
51	1
65	14
69	20
72	23
75	26
81	30

To get an insight of how these x and y values might relate to one another, we can plot them as coordinates along two axes, like this:



Now we're ready to apply an algorithm to our training data and fit it to a function that applies an operation to x to calculate y . One such algorithm is *linear regression*, which works by deriving a function that produces a straight line through the intersections of the x and y values while minimizing the average distance between the line and the plotted points, like this:



The line is a visual representation of the function in which the slope of the line describes how to calculate the value of y for a given value of x . The line intercepts the x axis at 50, so when x is 50, y is 0. As you can see from the axis markers in the plot, the line slopes so that every increase of 5 along the x axis results in an increase of 5 up the y axis; so when x is 55, y is 5; when x is 60, y is 10, and so on. To calculate a value of y for a given value of x , the function simply subtracts 50; in other words, the function can be expressed like this:

$$f(x) = x - 50$$

You can use this function to predict the number of ice creams sold on a day with any given temperature. For example, suppose the weather forecast tells us that tomorrow it will be 77 degrees. We can apply our model to calculate $77 - 50$ and predict that we'll sell 27 ice creams tomorrow.

But just how accurate is our model?

Evaluating a regression model

To validate the model and evaluate how well it predicts, we held back some data for which we know the label (y) value. Here's the data we held back:

Expand table

Temperature (x)	Ice cream sales (y)
52	0
67	14
70	23
73	22
78	26
83	36

We can use the model to predict the label for each of the observations in this dataset based on the feature

(x) value; and then compare the predicted label ($\hat{y}$) to the known actual label value (y).

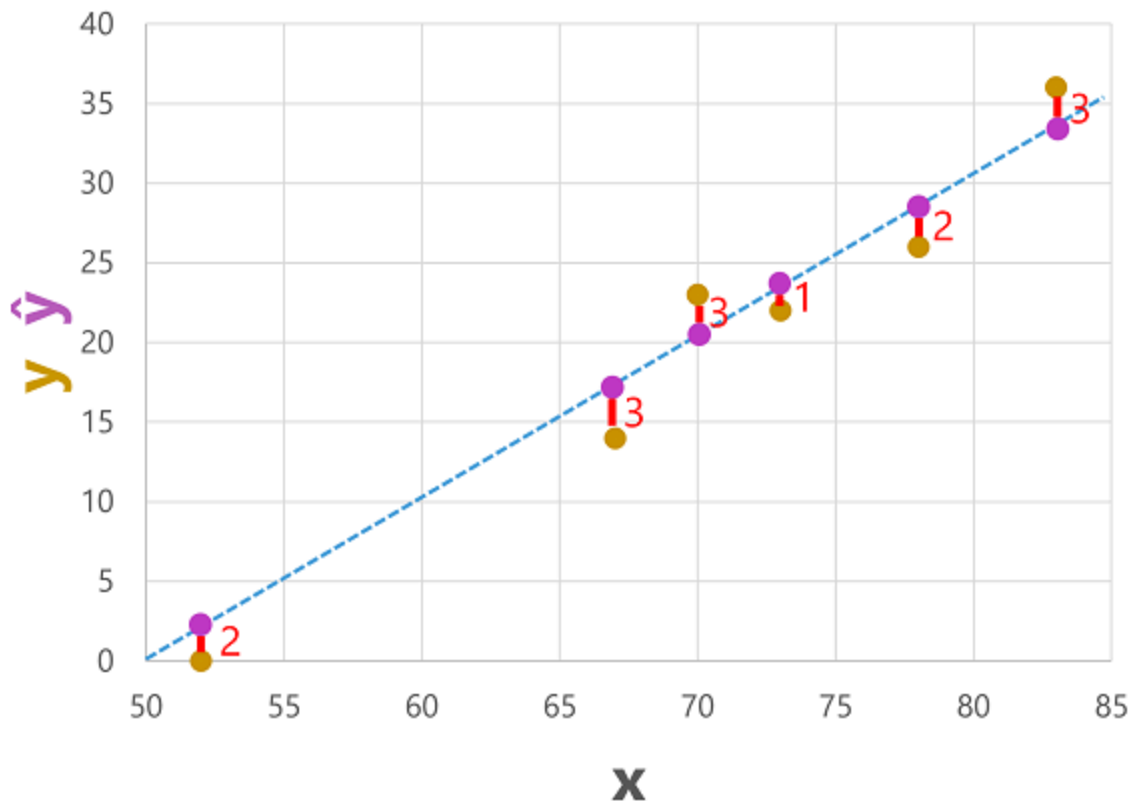
Using the model we trained earlier, which encapsulates the function $f(x) = x - 50$, results in the following predictions:

Expand table

Temperature (x)	Actual sales (y)	Predicted sales ($\hat{y}$)
52	0	2
67	14	17
70	23	20
73	22	23
78	26	28
83	36	33

We can plot both the predicted and actual labels against the feature values like this:

We can plot both the predicted and actual labels against the feature values like this:



The predicted labels are calculated by the model so they're on the function line, but there's some variance between the $\hat{y}$ values calculated by the function and the actual y values from the validation dataset; which is indicated on the plot as a line between the $\hat{y}$ and y values that shows how far off the prediction was from the actual value.

Regression evaluation metrics

Based on the differences between the predicted and actual values, you can calculate some common metrics that are used to evaluate a regression model.

Mean Absolute Error (MAE)

The variance in this example indicates by how many ice creams each prediction was wrong. It doesn't matter if the prediction was over or under the actual value (so

for example, -3 and +3 both indicate a variance of 3). This metric is known as the absolute error for each prediction, and can be summarized for the whole validation set as the mean absolute error (MAE). In the ice cream example, the mean (average) of the absolute errors (2, 3, 3, 1, 2, and 3) is 2.33.

Mean Squared Error (MSE)

The mean absolute error metric takes all discrepancies between predicted and actual labels into account equally. However, it may be more desirable to have a model that is consistently wrong by a small amount than one that makes fewer, but larger errors. One way to produce a metric that "amplifies" larger errors by squaring the individual errors and calculating the mean of the squared values. This metric is known as the mean squared error (MSE).

In our ice cream example, the mean of the squared absolute values (which are 4, 9, 9, 1, 4, and 9) is 6.

Root Mean Squared Error (RMSE)

The mean squared error helps take the magnitude of errors into account, but because it squares the error values, the resulting metric no longer represents the quantity measured by the label. In other words, we can say that the MSE of our model is 6, but that doesn't measure its accuracy in terms of the number of ice creams that were mispredicted; 6 is just a numeric score that indicates the level of error in the validation predictions.

If we want to measure the error in terms of the number of ice creams, we need to calculate the square root of the MSE; which produces a metric called, unsurprisingly, Root Mean Squared Error. In this case $\sqrt{6}$, which is 2.45 (ice creams).

My Example:

<https://github.com/PrajwalBS-coder/ML/tree/basic/Regression>

Coefficient of determination (R²)

All of the metrics so far compare the discrepancy between the predicted and actual values in order to evaluate the model. However, in reality, there's some natural random variance in the daily sales of ice cream that the model takes into account. In a linear regression model, the training algorithm fits a straight line that minimizes the mean variance between the function and the known label values. The coefficient of determination (more commonly referred to as R² or R-Squared) is a metric that measures the proportion of variance in the validation results that can be explained by the model, as opposed to some anomalous aspect of the validation data (for example, a day with a highly unusual number of ice creams sales because of a local festival).

The calculation for R² is more complex than for the previous metrics. It compares the sum of squared differences between predicted and actual labels with the sum of squared differences between the actual label values and the mean of actual label values, like this:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Don't worry too much if that looks complicated; most machine learning tools can calculate the metric for you. The important point is that the result is a value between 0 and 1 that describes the proportion of variance explained by the model. In simple terms, the closer to 1 this value is, the better the model is

fitting the validation data. In the case of the ice cream regression model, the R^2 calculated from the validation data is 0.95.